

EV Predictions System

Complete Git-Based Collaboration and MLOps Assignment Roadmap

Team: Faheem and Moeer

Repository: EV-Predictions-System

Purpose: Follow this document in order and preserve every commit, pull request, review, experiment, test result, and release artifact required by the assignment.

How to use this roadmap

The assignment is graded from the repository and Git history. Complete phases in order. Every phase has an owner, a reviewer, commands, and a checkpoint. Faheem owns the existing project foundation and data/pipeline work. Moeer owns onboarding, CI, notebook work, and release coordination. Both members must still code, author commits, open pull requests, review each other, and run the project locally.

Current project position

Already present	Still required
Modular source files, tests, params.yaml, pyproject.toml, uv.lock, README.md, CONTRIBUTING.md, pre-commit configuration, PR template, and main/dev/staging branches.	DVC setup and remote, tracked dataset, dvc.yaml, dvc.lock, EDA notebook, CI workflow, experiments, data-update PR, conflict PR, changes-requested review, REPORT.md, independent reproduction, and model-v1.0.

Roles and branch rules

Person	Primary ownership	Mandatory personal evidence
Faheem	Data and Hygiene: dataset, DVC, validation, pre-commit, environment, pipeline.	2+ merged PRs, 2+ reviews, 3+ experiments, own commits.
Moeer	Model Operations: onboarding, notebook, CI, experiments, release coordination.	2+ merged PRs, 2+ reviews, 3+ experiments, own commits.

Branch flow: exp/<member>-<idea> → feat/<name> or data/<name> → dev → staging → main. No direct pushes to dev, staging, or main after initial setup. Use Conventional Commits. Continue the current team decision: squash PRs into dev; preserve release history for staging and main.

Phase 1 - Moeer onboarding and repository access

Ownership: Faheem leads; Moeer performs the work

1. Moeer configures his own identity: `git config user.name "Moeer"` and `git config user.email "his-email"`.
2. Moeer updates the repository and creates `feat/moeer-onboarding`.
3. Moeer runs `uv sync`, `uv run pytest`, `uv run ruff check .`, and `uv run ruff format --check .`

4. Moeer makes a small useful improvement, such as a README setup section or test improvement, commits it, pushes the branch, and opens a PR into dev.
5. Faheem checks out the branch, runs the change, reviews it, and merges it.
6. Faheem confirms both members have collaborator access and the instructor is added as viewer. Protect main, staging, and dev with PR approval and force-push blocking.

Checkpoint: Moeer has his first authored commit, branch, pull request, and review. Both members can push branches.

Phase 2 - DVC dataset setup

Ownership: Faheem owns; Moeer verifies

1. Faheem creates `data/initial-dataset` from dev.
2. Faheem installs/initializes DVC, configures a shared remote, and keeps credentials only in `.dvc/config.local` or environment variables.
3. Faheem places the dataset at the path in `params.yaml` and validates required columns, target values, nulls, duplicates, and ranges.
4. Faheem runs `dvc add data/raw/<dataset>.csv`, then `dvc push` before committing or pushing Git changes.
5. Faheem opens a PR into dev with the DVC pointer and configuration.
6. Moeer clones/checks out the branch fresh, runs `dvc pull`, confirms the dataset arrives, and verifies the actual CSV is not in Git history.

Checkpoint: only DVC pointers are tracked by Git and Moeer can pull the actual data.

Phase 3 - Pre-commit and secret protection

Ownership: Faheem owns; Moeer tests

1. Faheem finalizes Ruff lint/format, nbstripout, 1 MB large-file blocking, and detect-secrets/Gitleaks.
2. Both members run `pre-commit install` and `pre-commit run --all-files`.
3. Moeer tests a 5 MB dummy file and fake API key; both must be blocked. Do not leave either in Git history.
4. Faheem opens the PR; Moeer reviews it and screenshots the blocked checks for the report.

Checkpoint: guard-rail screenshots exist and the reviewed PR is merged into dev.

Phase 4 - EDA notebook and tested reusable logic

Ownership: Moeer owns; Faheem reviews

1. Moeer creates `feat/moeer-eda-notebook`.
2. Moeer creates `notebooks/01-eda.ipynb` with dataset overview, missingness, target distribution, feature analysis, and useful plots.
3. Moeer creates the Jupyter pair using `uv run jupyter --set-formats ipynb,py:percent notebooks/01-eda.ipynb`.
4. Moeer moves one reusable cleaning/feature function into `src/` and adds a unit test.
5. Moeer restarts the kernel and runs every cell top-to-bottom. Outputs and execution counts must be stripped.
6. Faheem reviews the notebook, checks leakage and paths, runs it, and approves the PR.

Checkpoint: .ipynb and .py files are committed, reusable logic is tested, and the notebook diff is clean.

Phase 5 - Reproducible DVC pipeline

Ownership: Faheem owns; Moeer reproduces

1. Faheem creates feat/faheem-dvc-pipeline.
2. Faheem moves all paths, split ratios, hyperparameters, and seeds into configs/params.yaml.
3. Faheem defines prepare, train, and evaluate stages in dvc.yaml and generates dvc.lock.
4. Preprocessing must fit on training data only. Every randomness source must use a fixed seed.
5. evaluate writes metrics.json containing RMSE, MAE, R2, test count, parameters, seed, and current Git SHA.
6. Faheem runs `dvc repro`, `dvc push`, and opens a PR.
7. Moeer uses a fresh clone, runs `dvc pull && dvc repro`, and confirms identical metrics.

Checkpoint: the pipeline and metrics reproduce exactly from a fresh clone.

Phase 6 - Experiments and PR collaboration

Ownership: Both members separately

1. Faheem creates exp/faheem-model-tuning and runs at least three experiments, for example `max_depth` 6, 10, and 14.
2. Moeer creates exp/moeer-model-tuning and runs at least three experiments, for example `n_estimators` 50, 100, and 200.
3. Both run `dvc exp show`, save the comparison, and explain the selected winner.
4. Keep one experiment branch abandoned and explain why in REPORT.md.
5. Each member promotes their winner onto a clean feat/ branch and opens a PR with before/after metrics.
6. Each member must author 2+ merged PRs, review 2+ PRs, and request changes at least once.

Checkpoint: GitHub history shows both members as authors and reviewers, with experiment evidence.

Phase 7 - Data update and conflict resolution

Ownership: Faheem owns data; Moeer owns second conflict branch

1. Faheem creates data/update-dataset, makes a real dataset change, runs `dvc add` and `dvc push`, and opens a PR.
2. Moeer tests old and new data versions with `git checkout` and `dvc checkout` before approving.
3. Both members change the same params.yaml line on separate branches.
4. Merge the first PR. Moeer rebases his branch on dev, resolves the conflict, runs tests and `dvc repro`, and documents the resolution.
5. Faheem reviews the resolved PR.

Checkpoint: data-update and conflict-resolution PRs are visible and reproducible.

Phase 8 - Pull-request CI

Ownership: Moeer owns; Faheem reviews

1. Moeer creates feat/moeer-ci and adds .github/workflows/ci.yml for PRs targeting dev, staging, and main.
2. CI runs Ruff check, Ruff format check, pytest, schema/null/range checks, and a small smoke-training run.

3. Faheem reviews and runs the workflow.
4. Moeer intentionally breaks a test and captures the red blocked check, then fixes it and captures the passing check.
5. Make passing CI checks required in all three branch-protection rules.

Checkpoint: broken PRs cannot merge; valid PRs pass all required checks.

Phase 9 - Release and independent reproduction

Ownership: Moeer coordinates release; Faheem reproduces

1. Moeer opens release: v1.0 from dev into staging and summarizes included PRs and final metrics.
2. Faheem clones into a new folder and runs git checkout staging, uv sync, dvc pull, and dvc repro.
3. Faheem compares metrics.json exactly with the release PR and attaches evidence.
4. Merge staging, open staging → main, obtain approval, merge, and tag model-v1.0 on main.
5. Both members conduct a retrospective and record lessons in CONTRIBUTING.md.
6. Optional bonus: perform a real fix/ branch and release model-v1.0.1.

Checkpoint: model-v1.0 exists on main and independent reproduction succeeds.

Commands for the release tag

Run only after staging reproduction succeeds and the staging-to-main PR is approved:

```
git switch main git pull git tag -a model-v1.0 -m "First production model" git push origin model-v1.0
```

REPORT.md must contain

- Team members, roles, dataset source, and starter-code source.
- Branching and merge policy.
- Final reproducibility table: release SHA, params.yaml values, data DVC hash, uv.lock reference, seed, and metrics.
- dvc exp show comparison and winner rationale.
- Links to data-update, conflict-resolution, changes-requested, release PRs, and abandoned experiment branch.
- Screenshots of blocked large file/secret, failing CI, and passing CI.
- Independent reproduction commands and exact metric comparison.
- Retrospective and one contribution paragraph for both Faheem and Moeer.

Final submission audit

- [] Dataset/model are not committed to Git; DVC pointers and remote work.
- [] dvc.yaml and dvc.lock are committed.
- [] Fresh-clone dvc pull && dvc repro produces identical metrics.
- [] dev, staging, and main are protected.
- [] Faheem and Moeer each authored 2+ merged PRs and reviewed 2+ PRs.
- [] Each member completed 3+ experiments.
- [] Changes-requested, data-update, conflict, and abandoned experiment evidence exists.
- [] Notebook outputs are stripped and Jupyter text pair is committed.
- [] CI covers lint, format, tests, data checks, and smoke training.
- [] Failed and successful CI screenshots exist.
- [] REPORT.md is complete.
- [] model-v1.0 exists on main.

Immediate starting sequence

- 1. Moeer:** configure Git identity, create feat/moeer-onboarding, run the tests, make a small improvement, and open his first PR.
- 2. Faheem:** create data/initial-dataset, configure DVC, and open the data PR.
- 3. Review swap:** Faheem reviews Moeer's onboarding PR; Moeer reviews Faheem's DVC PR.
- 4. Continue:** Moeer starts CI and Faheem starts the DVC pipeline.